

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA1102

COURSE OUTCOME COVERED

C02: Design UML diagrams to represent system requirements and architecture.

(BL3)

Assessment Tool Weightage: 60%

QUESTIONS: 6

Total Marks:60

Annexure A

Application Based Questions

Code of Conduct:

I D Varshini / 192320007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Application Based Questions

This annexure presents structured, analytical solutions to six application-based UML modelling problems: a Smart Hotel Reservation System, a University Management System, a Vehicle Rental System, a Traffic Signal Control System, a Smart Healthcare Portal, and a Smart City Waste Management System. Each solution analyses the given scenario to identify missing actors, classes, objects, states, components, or nodes, and presents a complete, optimized UML diagram constructed to resolve the ambiguity described in the question.

1. Use Case Diagram Analysis — Smart Hotel Reservation System *[10 Marks]*

The original description only mentions guests, managers, and payment services, which blurs the boundary between front-desk operational duties and managerial duties. Analysing the scenario shows that "handling room allocation" is actually a day-to-day operational task better performed by front-desk staff, while true managerial responsibility lies in oversight and reporting. Distinguishing these roles clarifies actor responsibilities and prevents a single bloated "Manager" actor from absorbing operational use cases that do not belong to it.

Missing Actors and Use Cases Identified

- **Front Desk Staff** (missing actor) — separated from Manager to own Check-in/Check-out and day-to-day room allocation.
- **Duty Supervisor** (missing actor) — generalised from Manager, covers shift-level allocation decisions when the Manager is unavailable.
- **Register Guest Account** and **Login/Authenticate** (missing use cases) — the original design assumed guests could book without first authenticating, which is unrealistic for a system that stores payment and personal data.
- **Apply Discount/Promo Code** (missing use case) — an optional path noted as commonly present in modern reservation systems but absent from the original description.
- **Check-in Guest** and **Check-out Guest** (missing use cases) — essential operational use cases that were implied by "room allocation" but never made explicit.

Relationships Defined

Include: "Make Reservation" includes "Login/Authenticate" (mandatory identity check),

"Process Payment" (a reservation cannot be confirmed without payment), and "Send Booking Notification" (a confirmation is always generated). "Check-in Guest" includes "Login/Authenticate" so only verified staff can process check-ins.

Extend: "Cancel Booking" extends "Make Reservation" since cancellation is optional and conditional on an existing booking. "Apply Discount/Promo Code" extends "Make Reservation" as an optional enhancement. "Allocate Room" extends "Make Reservation" because automatic or manual room allocation is an optional refinement step that only applies under certain booking conditions.

Generalisation: "Duty Supervisor" is generalised from "Hotel Manager", inheriting the "Allocate Room" capability while manager-only tasks like "Generate Reports" remain restricted to the base Manager actor — this correction clearly separates strategic and operational responsibilities that the original design conflated.

Optimized Use Case Diagram

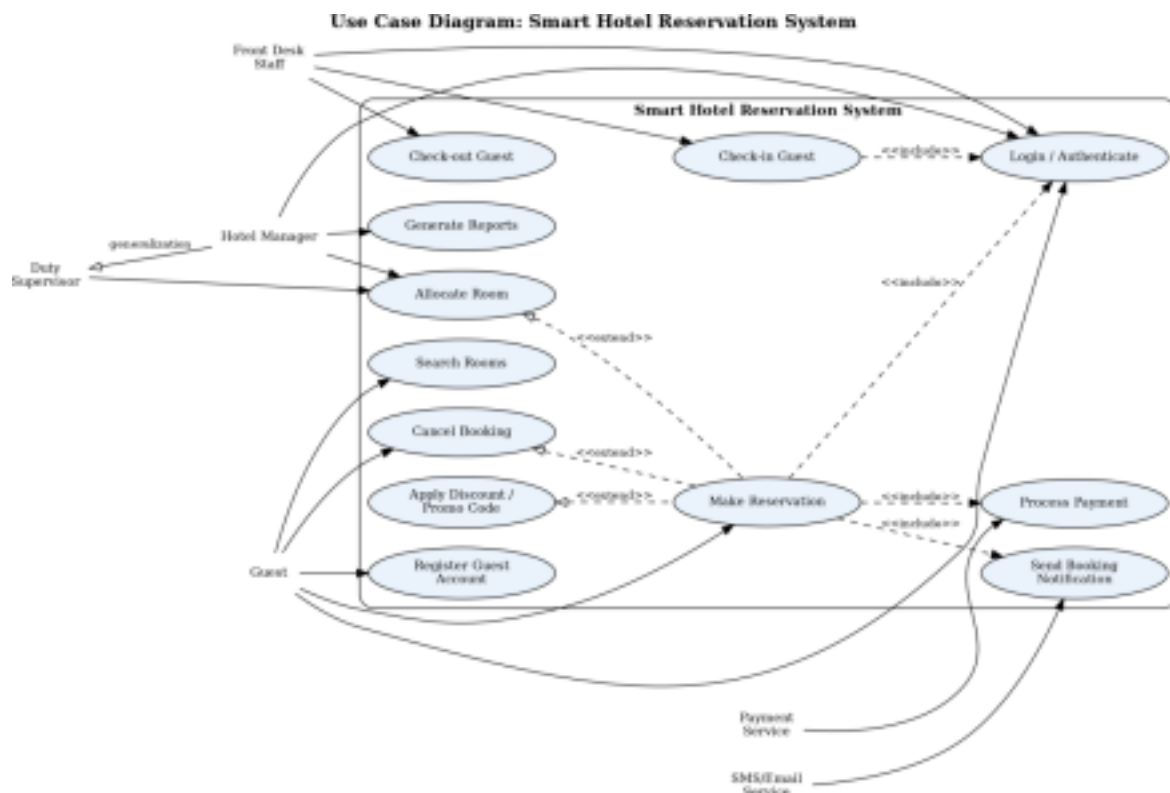


Figure 1: Optimized Use Case Diagram for the Smart Hotel Reservation System, with clearly separated actor responsibilities and include/extend/generalization relationships.

2. Class Diagram Design — University Management

System *[10 Marks]*

The scenario names five entities — Student, Faculty, Course, Department, and Examination — but gives no attributes, operations, or relationships. Analysis of a typical university workflow shows that Student and Faculty share common personal-identity data (a candidate for a common superclass), that Department organisationally owns Faculty and Course records, and that Course and Examination are tightly linked through the academic calendar.

Classes with Attributes and Methods

- **Person (abstract superclass):** personId, name, email, phone; getProfile(). •
Student (inherits Person): rollNumber, program, yearOfStudy, cgpa;
registerForCourse(course), viewResult(examId), payFees(amount).
- **Faculty (inherits Person):** facultyId, designation, specialization; assignCourse(course),
enterMarks(studentId, examId, marks), viewSchedule().
- **Department:** deptId, deptName, location; addFaculty(faculty), addCourse(course),
getDeptReport().
- **Course:** courseId, courseName, credits, semester; getSyllabus(), getEnrolledStudents(). •
Examination: examId, examDate, examType, maxMarks; scheduleExam(),
publishResult(), calculateGrade(marks).

Relationships Identified

Inheritance: Student and Faculty both inherit shared identity attributes from the abstract Person class, eliminating duplicate name/contact fields and allowing polymorphic handling of "any person in the university" where needed.

Aggregation: Department aggregates Faculty and Course — a department administratively owns these records, but a faculty member or course object is not destroyed if the department record is reorganised, so an open-diamond aggregation is appropriate rather than composition.

Association: Student and Course share a many-to-many "enrols in" association; Faculty and Course share a one-to-many "teaches" association; Course and Examination are associated because each course schedules one or more examinations; Student and Examination are associated through the "appears for" relationship recording which students sit which exams.

Class Diagram

Class Diagram: University Management System

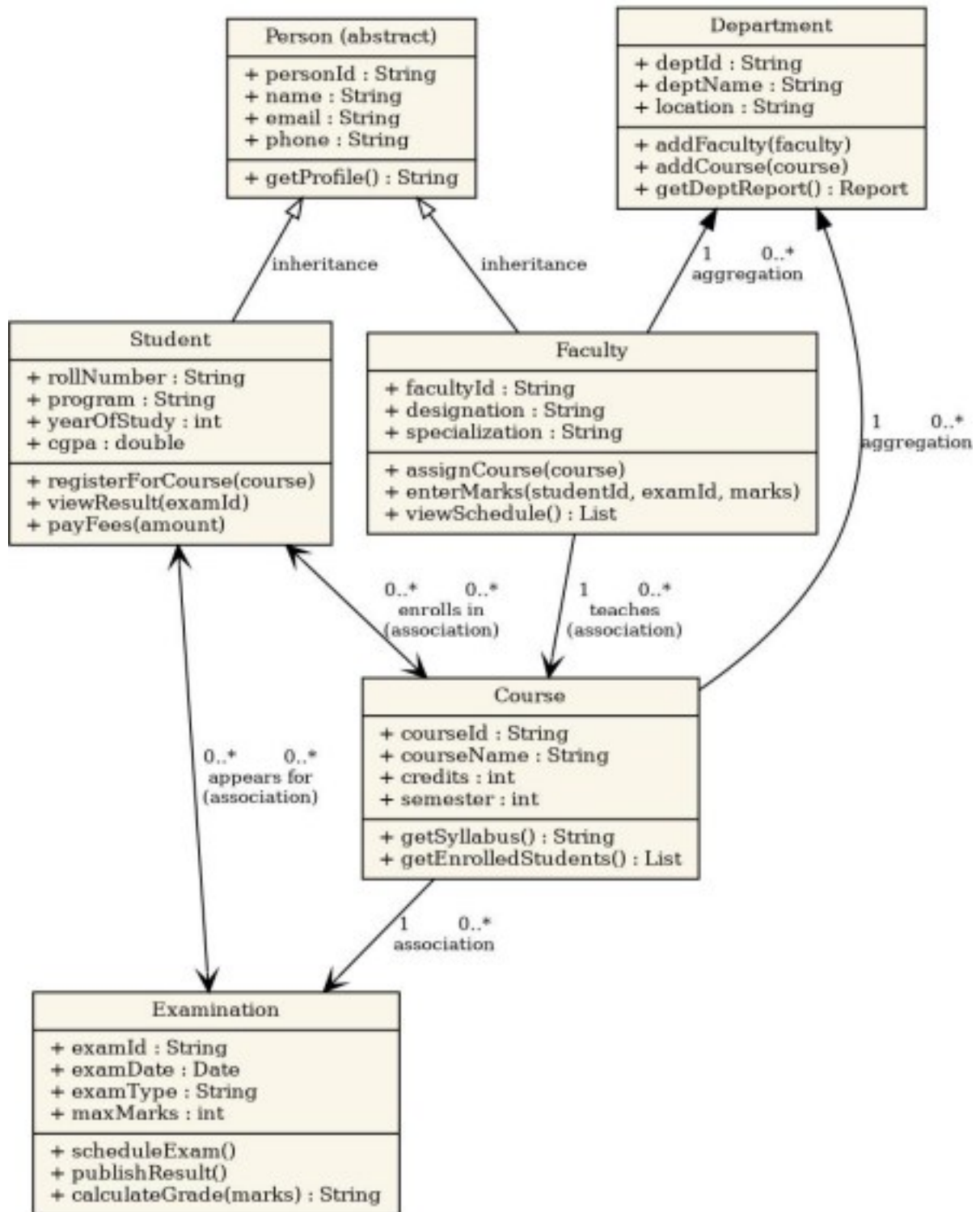


Figure 2: Class Diagram for the University Management System, showing inheritance, aggregation, association, attributes, methods, and multiplicities.

3. Sequence Diagram Evaluation — Vehicle Rental System

[10 Marks]

The given description mentions only four broad steps (search, book, pay, confirm), which omits the intermediate objects that actually coordinate availability checking and reservation locking. Evaluating the process flow shows that a booking cannot be safely confirmed without first verifying and reserving the vehicle in an inventory system — otherwise two customers could book the same vehicle simultaneously. This missing coordination step is corrected below.

Missing Objects Identified

- **:RentalController** — a control object coordinating booking logic; absent from the original description, which implied the UI spoke directly to payment and inventory.
- **:VehicleInventory** — an entity object that tracks real-time vehicle availability and performs the reservation lock; without it, the described flow cannot guarantee a vehicle is actually free.
- **:PaymentGateway** — the external payment-processing object, clearly implied but not explicitly named in the original scenario.

Corrected Message Sequence

The customer searches for a vehicle, and the UI delegates the query to the RentalController, which checks live availability with the VehicleInventory before returning matching results. Once the customer selects and books a vehicle, the RentalController validates the booking (a self-call) and reserves the specific vehicle in the inventory to prevent double-booking. Only after this reservation is confirmed does the system prompt for payment; the UI relays payment details to the PaymentGateway, and upon success, the RentalController finalises the booking and the confirmation cascades back to the customer.

Activation and Lifelines

The UI remains active throughout as the mediating boundary object. The RentalController's activation bar spans the validation and reservation phase, while VehicleInventory and PaymentGateway are activated only for their specific synchronous calls

(availability/reservation, and payment processing respectively), correctly reflecting that these are momentary, focused responsibilities rather than continuously running processes.

Detailed Sequence Diagram

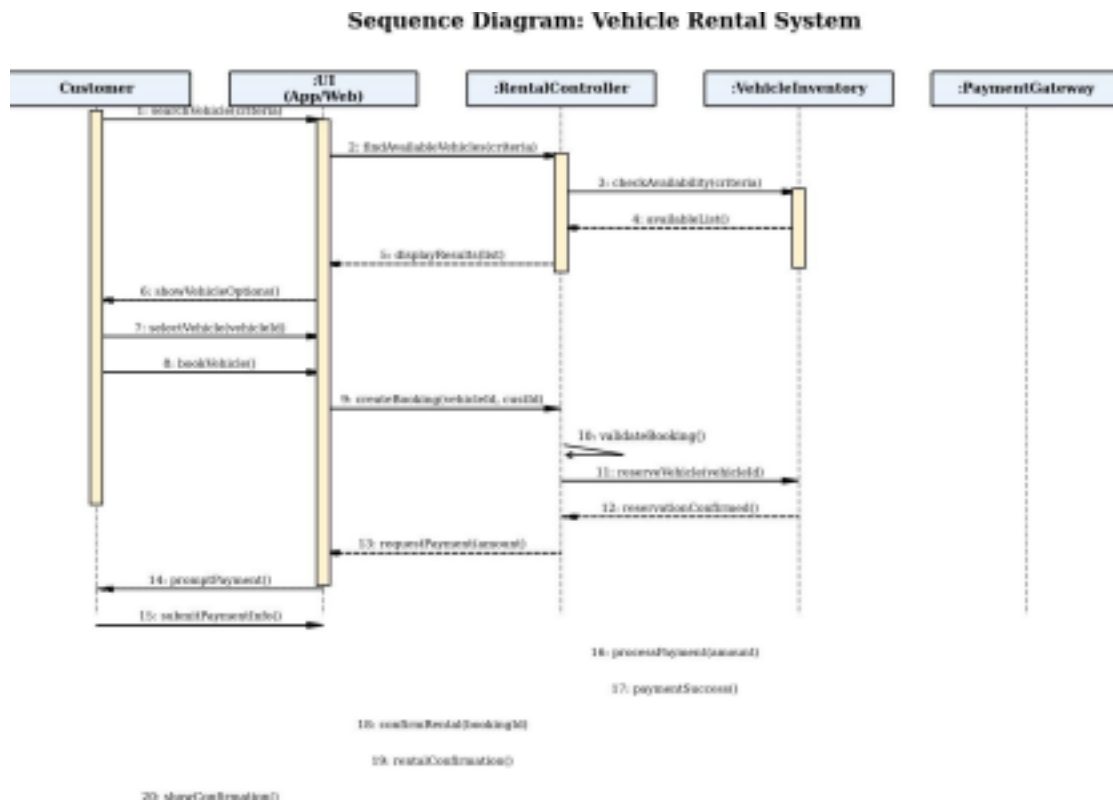


Figure 3: Sequence Diagram for the Vehicle Rental System, showing the corrected set of objects, availability-and-reservation coordination, and complete message flow through payment and confirmation.

4. State Diagram Construction — Traffic Signal Control

System [10 Marks]

The scenario lists five states but does not specify how or when the system moves between them. Analysing real traffic-signal behaviour shows that the normal Red→Green→Yellow→Red cycle is timer-driven, while Emergency Mode and Maintenance Mode are exception states that can be entered from any of the three normal states and must return control cleanly once the exceptional condition ends.

States and Transitions Identified

- **Red Signal** — default/initial state; halts traffic.
- **Green Signal** — entered after the red timer expires; allows traffic flow.
- **Yellow**

Signal — entered after the green timer expires; warns of an impending stop. •

Emergency Mode — can be entered from Red, Green, or Yellow the instant an emergency vehicle is detected, overriding the normal cycle.

- **Maintenance Mode** — can be entered from any normal state via a manual override by traffic control staff, and returns to Red Signal (a safe default) via `resumeAutoMode()`.

Events and Conditions Represented

Each normal-cycle transition is triggered by a `timerExpired()` event, ensuring the signal advances automatically at fixed intervals. The interrupt transitions into Emergency Mode are triggered by an `emergencyVehicleDetected()` event and guarded so that, on `emergencyCleared()`, control returns specifically to Green Signal for the priority lane rather than resuming mid-cycle, which would be unsafe. Maintenance Mode is entered via a `manualOverride()` event from any state and exited via `resumeAutoMode()`, with an optional `systemShutdown()` event leading to the final state for planned power-down.

Initial and Final States

A filled-circle initial pseudostate transitions directly into Red Signal when the controller powers on, reflecting the safest default state. A final state is reached only from Maintenance Mode via an explicit `systemShutdown()` event, representing a deliberate administrative power-off rather than a state the signal reaches during ordinary operation.

State Transition Diagram

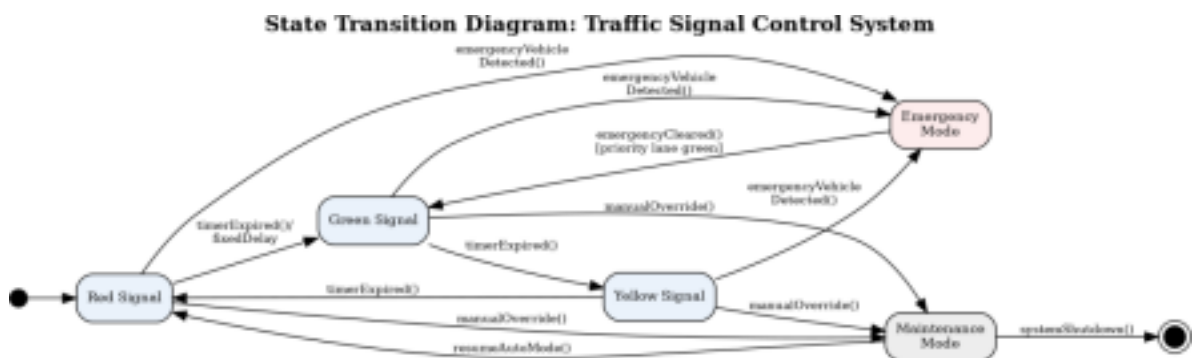


Figure 4: State Transition Diagram for the Traffic Signal Control System, showing the normal timer-driven cycle plus emergency and maintenance interrupt states.

5. Component Diagram Design — Smart Healthcare

Portal *[10 Marks]*

The five named modules — Patient Management, Appointment Scheduling, Medical Records, Billing, and Notification Services — are described only as a flat list with no interaction detail. Analysing the architecture shows clear dependency directions: scheduling and clinical modules must verify a patient's identity before acting, and billing depends on confirmed appointments, which was previously left undefined.

Major Components and Interactions

- **Patient Management** — owns core patient identity and demographic data; exposes IPatient.
- **Appointment Scheduling** — manages booking of doctor appointments; exposes IAppointment; depends on Patient Management to confirm patient identity.
- **Medical Records** — stores clinical history and visit notes; exposes IRecord; depends on Patient Management for patient linkage.
- **Billing Module** — generates invoices; exposes IBilling; depends on Appointment Scheduling to know which billable services were rendered.
- **Notification Services** — sends reminders and billing alerts; exposes INotify; used by both Appointment Scheduling and Billing.

Dependencies and Interfaces

Each module exposes its functionality strictly via a provided interface consumed by the client UI, so any module can be reimplemented internally without breaking others. The dependency chain runs from Billing → Appointment Scheduling → Patient Management, and separately Medical Records → Patient Management, meaning Patient Management acts as the foundational identity component that all clinical and administrative modules rely on. All modules persist data to a shared database component.

Component Diagram

Component Diagram: Smart Healthcare Portal

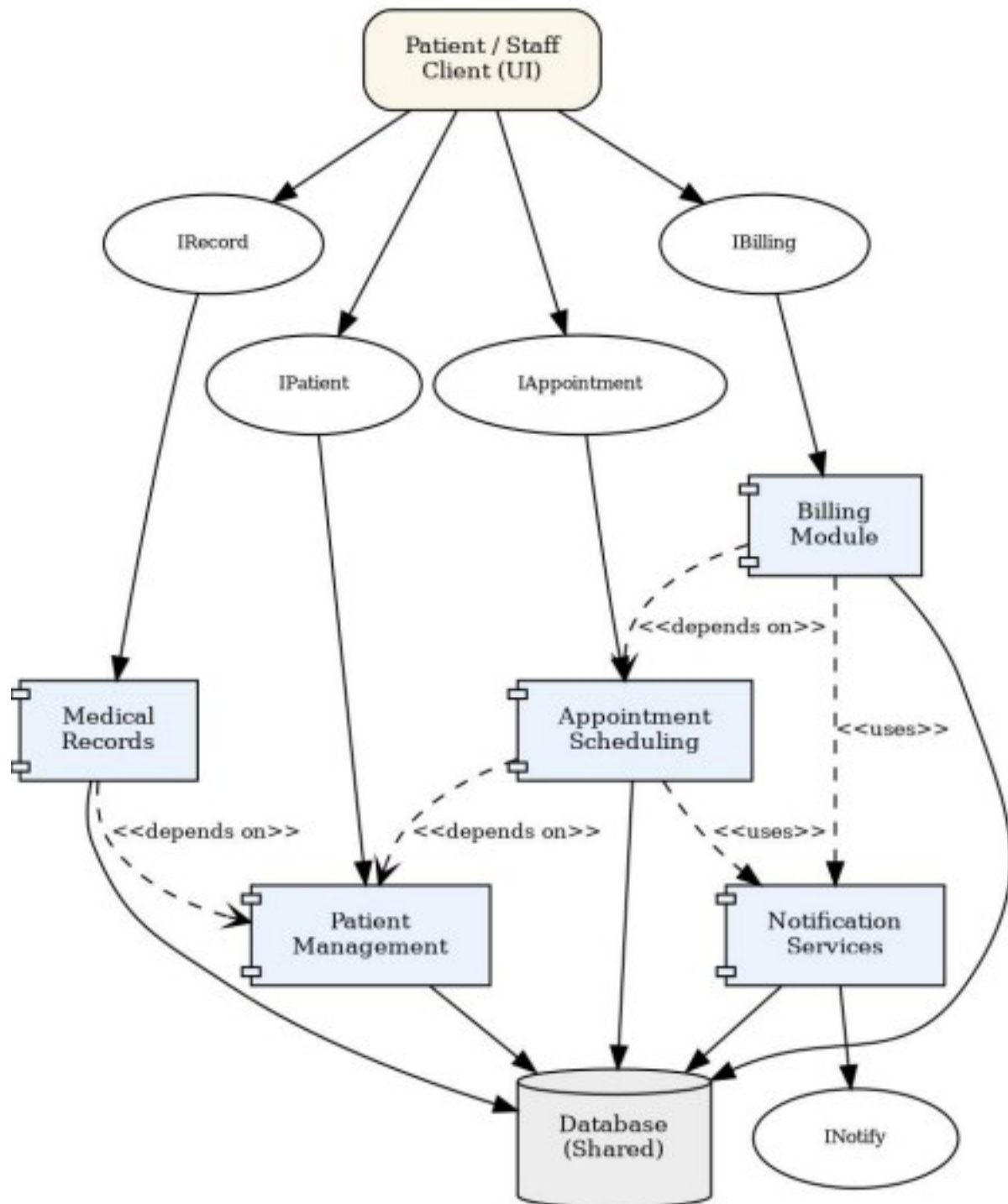


Figure 5: Component Diagram for the Smart Healthcare Portal, showing components, provided interfaces, and inter-module dependencies.

6. Deployment Diagram Design — Smart City Waste Management

System *[10 Marks]*

The scenario names four elements — IoT waste-bin sensors, edge devices, cloud servers, and monitoring dashboards — but does not specify what artifacts run on each node or how they communicate. Analysing the deployment environment shows a typical three-tier IoT architecture: field-level sensing, edge-level aggregation, and cloud-level analytics/storage, plus client devices for both monitoring staff and collection-vehicle drivers.

Nodes, Artifacts, and Communication Links

- **IoT Waste-Bin Sensor Unit** (device): runs Fill-Level Sensor Firmware and a GPS/Location Module; connects to the Edge Gateway over low-power wireless protocols such as LoRaWAN or Zigbee.
- **Edge Gateway Node** (device): runs a Data Aggregation & Filtering Agent that batches and cleans sensor readings before forwarding them; connects to the Cloud Server over HTTPS/MQTT via a 4G/5G uplink.
- **Cloud Server** (execution environment): hosts the Waste Data API Service, Route Optimization Engine, Alert Generation Service, and the central Waste Database; internal components communicate via internal calls and JDBC.
- **Monitoring Workstation** (device): runs a browser-based Monitoring Dashboard that consumes the API over HTTPS and receives real-time alerts from the Alert Generation Service.
- **Collection Vehicle Device** (device): runs a Driver Navigation App that receives optimized collection routes as push notifications from the Route Optimization Engine.

Physical Architecture Summary

Data flows from thousands of distributed bin sensors up through nearby edge gateways, which reduce transmission volume before reaching the cloud tier. The cloud tier performs the computationally heavier tasks — route optimisation and alerting — and serves both a human-facing dashboard for city monitoring staff and a machine-facing push channel for drivers, ensuring the physical deployment matches the differing bandwidth, power, and processing constraints of each tier.

Deployment Diagram



Figure 6: Deployment Diagram for the Smart City Waste Management System, showing physical nodes, deployed artifacts, and communication links across the sensor, edge, and cloud tiers.

RUBRICS FOR CALCULATION

Application Based Questions

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand